

Deploy & Secure

an Azure Linux VM

LINUX · SSH · NSG · UFW · FAIL2BAN · AZURE SECURITY

OBJECTIVE Deploy an Ubuntu Linux virtual machine on Microsoft Azure and apply a layered security hardening approach: locking SSH access to a single IP via NSG, disabling password authentication in favor of key-based auth, configuring UFW firewall rules, installing fail2ban for brute force protection, and enabling automatic security updates.

Lab Environment

Component	Details
Platform	Microsoft Azure (Free Subscription)
VM Name	vm-001
OS	Ubuntu 24.04 LTS (Linux)
Auth Method	SSH key pair (no password login)
Access Control	NSG restricted to personal computer IP only
Firewall	UFW (Uncomplicated Firewall) + fail2ban

Security Controls Applied

✓	NSG Rule	SSH (port 22) restricted to single personal IP address
✓	SSH Key Auth	Password authentication disabled — key-pair only
✓	UFW Firewall	Host-based firewall enabled, only SSH allowed
✓	fail2ban	Auto-bans IPs after repeated failed login attempts
✓	Auto Updates	Unattended security upgrades enabled
✓	OS Hardening	sudo apt update && upgrade on initial deployment

Create a Resource Group

All Azure resources must live inside a resource group. This keeps everything organized and makes cleanup easy — delete the group to remove everything at once.

A Navigate to Resource Groups

- In the Azure Portal, search 'Resource Groups' in the top search bar
- Click + Create

B Configure and create

- Name: az-vm-ssh (or any name)
- Region: choose a region close to you
- Click Review + Create → Create

- Keep all resources in the same region to avoid latency and connectivity issues.

Deploy the Ubuntu Linux VM

A Create the Virtual Machine

- Search 'Virtual Machines' → Click Create → Azure Virtual Machine
- Resource Group: az-vm-ssh | VM Name: vm-001
- Region: same as resource group
- Image: Ubuntu Server 24.04 LTS
- Size: Standard_B1s (free tier eligible)

B Configure authentication — SSH key pair

- Authentication type: SSH public key
- Username: azureuser (or your choice)
- SSH public key source: Generate new key pair
- Azure will generate a .pem key file — download and save it securely
- ■■ If you lose this key file you cannot reconnect to the VM

C Inbound port rules

- Public inbound ports: Allow selected ports
- Select: SSH (22)
- Click Review + Create → Create (~2 minutes to deploy)

Restrict SSH Access via NSG Rule

By default Azure allows SSH from any IP. This is a major security risk. We lock it down so only your personal computer can connect.

A Find your public IP address

- On your personal computer, go to whatismyip.com
- Copy your IPv4 address — this is what you'll whitelist

B Edit the NSG inbound SSH rule

- In your resource group, click `vm-001-nsg`
- Click Inbound security rules → find the SSH rule → click it to edit
- Source: IP Addresses
- Source IP address: [your personal IP]
- Destination port: 22 | Protocol: TCP | Action: Allow | Priority: 100
- Click Save

■ Now only your IP can SSH into this VM. Anyone else hitting port 22 will be silently dropped by the NSG before they even reach the server.

Connect to the VM via SSH

A Get the VM public IP

- In the Azure Portal, open your `vm-001`
- The public IP is displayed on the Overview page

B Connect from your terminal or CMD

- Open Command Prompt, PowerShell, or Terminal
- Navigate to where you saved the `.pem` key file
- Run the SSH command:

```
ssh -i "C:\Users\YourName\Downloads\vm-001_key.pem" azureuser@
```

C Accept the host fingerprint

- First connection will ask: 'Are you sure you want to continue connecting?'
- Type yes and press Enter
- You are now inside the Ubuntu VM

Update the Operating System

Always run OS updates immediately after deploying a new server. This patches known vulnerabilities before anything else is installed.

A Run system update and upgrade

→ This downloads the latest package lists and installs all pending security patches

```
sudo apt update && sudo apt upgrade -y
```

■ This may take 1-3 minutes on a fresh VM. The -y flag auto-accepts all prompts.

Configure UFW Host Firewall

UFW (Uncomplicated Firewall) is a host-based firewall that runs inside the VM — an additional layer on top of the Azure NSG. Defense in depth.

A Install and configure UFW

→ Install UFW, allow SSH, then enable the firewall:

```
sudo apt install ufw -y
sudo ufw allow ssh
sudo ufw enable
```

■■ Always run 'allow ssh' BEFORE 'enable' or you risk locking yourself out.

B Verify firewall status

→ Confirm UFW is active and SSH is allowed:

```
sudo ufw status

# Expected output:
# Status: active
# To Action From
# --
# 22/tcp ALLOW Anywhere
# 22/tcp (v6) ALLOW Anywhere (v6)
```

Disable Password Authentication

Even with the NSG locked to your IP, disabling password auth is best practice. Key-based authentication is cryptographically stronger and eliminates password brute force entirely.

A Edit the SSH daemon config

→ Open the SSH config file with nano editor:

```
sudo nano /etc/ssh/sshd_config
```

B Change these three settings

→ Find each line and change/verify the values below

→ Use Ctrl+W in nano to search for each setting:

```
PasswordAuthentication no
ChallengeResponseAuthentication no
UsePAM yes
```

C Save and exit nano

→ Press Ctrl+X → Y → Enter to save and close

D Restart SSH to apply changes

→ Restart the SSH service to load the new config:

```
sudo systemctl restart sshd
```

■ Password logins are now disabled. Only your .pem key can authenticate. Attackers cannot brute force a password that doesn't exist.

Install fail2ban — Brute Force Protection

fail2ban monitors auth logs and automatically bans IPs that have too many failed login attempts. Even with key auth, this protects against SSH scanning and enumeration.

A Install and enable fail2ban

→ Install fail2ban and enable it to start automatically on boot:

```
sudo apt install fail2ban -y
sudo systemctl enable fail2ban --now
```

B Verify fail2ban is running

→ Check the service status:

```
sudo systemctl status fail2ban

# Should show: active (running)
```

■ Default config bans IPs for 10 minutes after 5 failed attempts. You can customize thresholds in `/etc/fail2ban/jail.local`

Enable Automatic Security Updates

Unattended upgrades automatically installs security patches without manual intervention — critical for keeping the server secure long-term.

A Install and configure unattended-upgrades

→ Install the package and launch the interactive configuration dialog:

```
sudo apt install unattended-upgrades -y
sudo dpkg-reconfigure --priority=low unattended-upgrades
```

B Select Yes in the dialog

→ The ncurses dialog will ask if you want to enable automatic updates

→ Select Yes and press Enter

■ Security patches will now install automatically. Critical CVEs get patched without waiting for manual maintenance windows.

Project Summary

Successfully deployed and hardened an Ubuntu 24.04 Linux VM on Microsoft Azure using a layered security approach. Each control addresses a specific threat vector:

✓	NSG IP Restriction	Blocks all SSH access except from your personal IP — eliminates external scanning exposure
✓	SSH Key Authentication	Removes the password attack surface entirely — keys are cryptographically unguessable
✓	UFW Firewall	Host-based layer blocking unexpected traffic even if NSG rules change
✓	Password Auth Disabled	Even if a port is reached, no password means no brute force vector
✓	fail2ban	Auto-bans repeated failed attempts — protects against enumeration and scanning
✓	Automatic Updates	Patches CVEs without manual intervention — closes known vulnerabilities continuously

Quick Command Reference

Command	Purpose
<code>sudo apt update && sudo apt upgrade -y</code>	Update OS and all packages
<code>sudo apt install ufw -y</code>	Install UFW firewall
<code>sudo ufw allow ssh && sudo ufw enable</code>	Allow SSH and activate firewall
<code>sudo ufw status</code>	Check firewall rules and status
<code>sudo nano /etc/ssh/sshd_config</code>	Edit SSH daemon configuration
<code>sudo systemctl restart sshd</code>	Reload SSH config after changes
<code>sudo apt install fail2ban -y</code>	Install brute force protection
<code>sudo systemctl enable fail2ban --now</code>	Enable and start fail2ban
<code>sudo systemctl status fail2ban</code>	Verify fail2ban is running